

INT315 · CLUSTER COMPUTING

UNIT II

Introduction to Programming in Scala

Features, Types, Operators, Type Inference, Collections & Functions

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Seven syllabus topics, fully covered with heavy code examples, a closing quiz and hands-on exercise

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit II

Table of Contents

Unit II at a Glance	4
1. Features of Scala	5
1.1 Runs on the JVM — Full Java Interoperability	5
1.2 Object-Oriented AND Functional, at Once.....	6
1.3 Statically Typed, with Type Inference	6
1.4 Conciseness and Expressiveness.....	7
1.5 Pattern Matching	7
1.6 Scalability — the Feature the Name Refers To	7
2. Basic Data Types and Literals Used in Scala	9
2.1 The Numeric Types	9
2.2 Boolean and Character	9
2.3 Strings and String Interpolation	10
2.4 Unit, Null, and Nothing — Scala's Special Types.....	10
3. Operators and Methods Used in Scala	12
3.1 Arithmetic, Relational, and Logical Operators.....	12
3.2 Operators Are Just Method Calls	12
3.3 Common Methods on Core Types	13
3.4 Method Call Syntax — With and Without Dots/Parens.....	13
4. Introduction to Type Inference	15
4.1 How It Works	15
4.2 Type Inference for Function Return Types	16
4.3 Type Inference With Generics and Collections	16
5. Mutable vs. Immutable Collections.....	18
5.1 val vs. var — Immutable and Mutable References.....	18
5.2 Immutable Collections — the Default in Scala	19
5.3 Mutable Collections — Opt-In, Explicit.....	19
5.4 Why Immutability Is the Default, Not an Afterthought	19
6. Functions in Scala.....	21
6.1 Defining Functions with def	21
6.2 Anonymous Functions (Lambdas).....	21
6.3 Higher-Order Functions	22
6.4 Currying.....	23
6.5 Recursion	24

7. Lists, Maps, and Streams in Scala	26
7.1 List — an Ordered, Immutable Linked Sequence	26
7.2 Map — Key-Value Pairs.....	27
7.3 Stream / LazyList — Lazily Evaluated Sequences.....	28
Unit II Review	31

Unit II at a Glance

Unit I introduced Apache Spark and mentioned, almost in passing, that Spark exposes APIs in Scala, Python, Java, and R. That passing mention undersells something important: Scala isn't just ONE option among several for Spark — it's the language Spark itself is written in. Spark's Driver, its DAG Scheduler, its Catalyst optimizer, its RDD and DataFrame internals are all Scala code running on the JVM. Learning Scala now means you'll be reading Spark's own source code, its official documentation examples, and its error messages in the language they were actually written in — not a translation.

This unit covers Scala from first principles: what makes it a distinctive language (a fusion of object-oriented and functional programming), its type system and literals, its operators, the compiler's type inference, the mutable/immutable distinction that runs through everything in Scala, functions as first-class values, and the three collection types you'll use constantly — List, Map, and Stream/LazyList. Every topic follows the same five-beat rhythm as Unit I: Theory & Mechanics, a Comic/Visual metaphor, Industry Context, Commands/Code you can run yourself, and a Self-Assessment — and this unit leans even more heavily on runnable code, since a programming language is best learned by reading and writing it.



Learning the Native Tongue

If Unit I was learning that a certain factory (Spark) exists and what it can build, Unit II is learning the actual language spoken on the factory floor. You could hire a translator (use PySpark, letting Python stand between you and Spark's real internals) and still get real work done -- plenty of people do. But once you can read the floor manager's own handwriting (Scala), debugging a stuck job, reading a stack trace, or understanding WHY an API behaves a certain way all become dramatically easier.

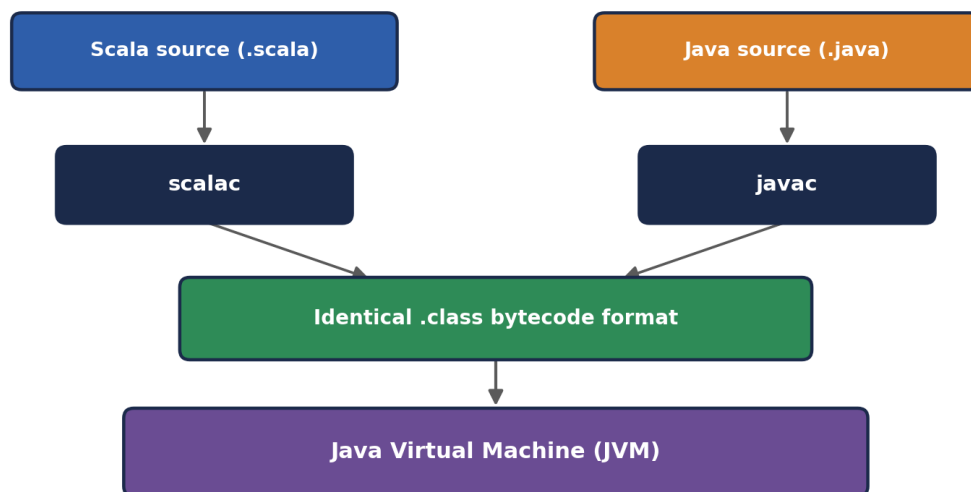
1. Features of Scala

Scala (a portmanteau of "scalable language") was created by Martin Odersky, first released in 2004, specifically to fuse two programming paradigms that were traditionally kept apart: object-oriented programming and functional programming. Its guiding design philosophy is that these two styles aren't rivals -- they're complementary tools, and a language that supports both well lets a programmer reach for whichever fits the problem at hand.

1.1 Runs on the JVM — Full Java Interoperability

Scala compiles down to the exact same .class bytecode format the Java compiler (javac) produces. This isn't an implementation detail you can ignore -- it's one of Scala's most practically important features: any existing Java library can be used directly from Scala code with no wrapper or adapter layer, and Scala classes can be called from Java code just as naturally.

Scala on the JVM — Full Interoperability with Java



Scala classes can call Java classes, and Java classes can call Scala classes, freely -- both are just bytecode to the JVM

Scala on the JVM — Full Interoperability with Java

```
// Using a standard Java class directly from Scala -- no import gymnastics
needed
import java.util.ArrayList

val javaList = new ArrayList[String]()
javaList.add("hello")
javaList.add("world")
println(javaList) // [hello, world]
```

```
// Java's String methods work exactly as you'd expect, because a Scala
// String literal IS a java.lang.String under the hood
val s = "Apache Spark"
println(s.toUpperCase)    // APACHE SPARK (a java.lang.String method)
println(s.length)        // 12
```

Why this specific feature mattered for Spark's own history

When Spark was created at UC Berkeley's AMPLab, choosing Scala meant the team could build directly on top of the entire mature Java ecosystem -- Hadoop's own client libraries, existing serialization frameworks, networking libraries -- while still writing the actual Spark engine in a more expressive, safer language than Java itself. That combination (Java's ecosystem, a better language on top) is a large part of why Scala was chosen for Spark's implementation in the first place.

1.2 Object-Oriented AND Functional, at Once

In Scala, EVERY value is an object -- there are no primitive types that live outside the object system the way Java's `int` or `boolean` do (Scala's `Int` and `Boolean` are objects too, though the compiler optimizes them down to JVM primitives under the hood for performance). At the same time, Scala treats functions as first-class values: a function can be stored in a variable, passed as an argument, and returned from another function, exactly like any other value. Section 6 explores this in depth.

```
// Object-oriented side: classes, inheritance, traits (Scala's version of
// interfaces)
abstract class Animal(val name: String) {
  def speak(): String
}
class Dog(name: String) extends Animal(name) {
  def speak(): String = s"$name says Woof!"
}
val d = new Dog("Rex")
println(d.speak())    // Rex says Woof!

// Functional side: the SAME language, a function as a value
val speak: String => String = name => s"$name says Woof!"
println(speak("Rex")) // Rex says Woof!
```

1.3 Statically Typed, with Type Inference

Every value in Scala has a type known at COMPILE time (unlike Python or JavaScript, where types are checked at runtime) -- this catches an entire category of bugs before the program ever runs. Yet Scala rarely feels as verbose as other statically typed languages like Java, because its compiler can usually INFER the type of a value from context rather than requiring it to be written explicitly. Section 4 covers type inference in full depth.

1.4 Conciseness and Expressiveness

Scala programs are frequently a fraction of the length of equivalent Java programs for the same logic -- Unit I's word-count comparison (60+ lines of Java MapReduce vs. 5 lines of Spark/Scala-style code) is a direct example of this same feature applied to Big Data specifically.

```
// A complete, self-contained data class in Scala -- one line
case class Employee(name: String, salary: Double, department: String)

// That single line automatically generates: a constructor, getters,
// a working equals()/hashCode(), a toString(), and pattern-matching support.
// The equivalent in Java is commonly 40-60 lines of boilerplate.

val emp = Employee("Aditi", 85000.0, "Engineering")
println(emp) // Employee(Aditi,85000.0,Engineering)
println(emp.copy(salary = 90000.0)) // Employee(Aditi,90000.0,Engineering)
```

1.5 Pattern Matching

Scala's match expression is a dramatically more powerful version of a switch statement -- it can match on types, structure, and conditions all at once, and (unlike a traditional switch) it's an EXPRESSION that produces a value.

```
def describe(x: Any): String = x match {
  case 0          => "zero"
  case n: Int if n < 0 => "a negative integer"
  case n: Int      => "a positive integer"
  case s: String   => s"a string of length ${s.length}"
  case List(a, b)  => s"a two-element list: $a and $b"
  case _          => "something else entirely"
}

println(describe(42)) // a positive integer
println(describe("spark")) // a string of length 5
println(describe(List(1, 2))) // a two-element list: 1 and 2
```

1.6 Scalability — the Feature the Name Refers To

The name "Scala" refers to the language's design goal of scaling smoothly from small scripts to massive systems using the SAME core language constructs -- the language doesn't force a rewrite or a paradigm shift as a codebase grows from a five-line script into a distributed system running across thousands of machines. Apache Spark itself, and Apache Kafka (also written substantially in Scala), are the two most visible proofs of this design goal succeeding in practice.

 The Swiss Army Knife vs. the Single-Purpose Tool

A language like pure Java pushes you firmly toward one paradigm (object-oriented); a language like pure Haskell pushes you firmly toward another (functional). Scala is a Swiss Army knife -- open the OOP blade when you're modeling a domain of related entities with inheritance, open the functional blade when you're transforming a stream of data through a pipeline of pure functions, and use both blades on the SAME class in the SAME file when that's genuinely the right call.

This is precisely why Spark's own internal codebase moves fluidly between defining classes (RDD, DataFrame, Dataset are all classes) and passing functions around as values (map, filter, reduce all take function arguments) -- the language doesn't force a choice between the two styles.

 Check yourself — Section 1

- Q1. Why can a Scala program call Java library code with zero adapter or wrapper layer?
- Q2. In your own words, what does it mean for a language to be "multi-paradigm," and name Scala's two main paradigms.
- Q3. What does a Scala `case class` automatically generate for you that you'd otherwise have to write by hand?

Hands-on Challenge:

Install Scala (or use an online REPL such as ScaStie) and define your own case class with at least three fields representing something from your own life (e.g., a Book, a Movie, or a Course). Create one instance, print it, then use `.copy()` to create a second instance that differs in exactly one field.

2. Basic Data Types and Literals Used in Scala

Every value manipulated by a Scala program has a type, and Scala's built-in types form a small, carefully designed hierarchy. This section covers the numeric, textual, and boolean types, and the LITERAL syntax used to write values of each type directly in source code.

2.1 The Numeric Types

Type	Description & Range
Byte	8-bit signed integer, -128 to 127
Short	16-bit signed integer, -32,768 to 32,767
Int	32-bit signed integer — the default for whole-number literals
Long	64-bit signed integer — literal must be suffixed with L
Float	32-bit single-precision floating point — literal suffixed with F
Double	64-bit double-precision floating point — the default for decimal literals

```
val b: Byte = 100
val s: Short = 30000
val i: Int = 2_000_000      // underscores are allowed as digit separators
val l: Long = 90000000000L  // the trailing L is REQUIRED for Long literals
val f: Float = 3.14F        // the trailing F is REQUIRED for Float literals
val d: Double = 3.14159265  // Double is the default -- no suffix needed

// Different numeric literal bases
val hex = 0xFF              // hexadecimal -> 255
val binary = 0b1010         // binary -> 10 (Scala 2.13+)
```

2.2 Boolean and Character

```
val isActive: Boolean = true
val hasError: Boolean = false

val grade: Char = 'A'      // single quotes for a single character
val newline: Char = '\n'
val unicodeHeart: Char = '\u2764'
```

2.3 Strings and String Interpolation

Scala's String type is `java.lang.String` directly (per Section 1.1's JVM interoperability), but Scala adds significantly more convenient syntax on top, especially STRING INTERPOLATION, which lets you embed variables and expressions directly inside a string literal.

```
val name = "Spark"
val version = 3.5

// s-interpolator: embed variables directly with $
println(s"Welcome to Apache $name $version")
// Welcome to Apache Spark 3.5

// full expressions need curly braces
println(s"Two plus two is ${2 + 2}")
// Two plus two is 4

// f-interpolator: adds printf-style formatting
println(f"Pi is approximately ${3.14159}%.2f")
// Pi is approximately 3.14

// raw-interpolator: does NOT process escape sequences
println(raw"A tab looks like \t here")
// A tab looks like \t here    (literal backslash-t, not an actual tab)

// Multi-line strings with triple quotes
val query = """
|SELECT customer_id, SUM(amount)
|FROM orders
|GROUP BY customer_id
""".stripMargin
```

2.4 Unit, Null, and Nothing — Scala's Special Types

- Unit — Scala's equivalent of Java's void; represents "no meaningful value," written as `()`. A function with no useful return value returns Unit, not null.
- Null — a trait whose only instance is the value null, itself an instance of any reference type; used sparingly in idiomatic Scala, which prefers Option (introduced in later units) to represent "possibly no value" safely.
- Nothing — a type with NO values at all; used as the return type of a method that never returns normally (e.g., one that always throws an exception), and sits at the very bottom of Scala's type hierarchy, making it compatible with every other type.

```
def logMessage(msg: String): Unit = {
  println(s"[LOG] $msg")
}
```

```
// implicitly returns (), Unit's only value
}

def fail(message: String): Nothing = {
  throw new RuntimeException(message)
}
```



A Filing Cabinet With Labeled Drawers

Think of Scala's type system as a filing cabinet where every drawer is labeled precisely: the `Byte` drawer only fits small numbers, the `Long` drawer fits huge ones, the `Char` drawer fits exactly one character. Unlike a loosely-typed language where you could stuff anything into any drawer and sort it out later, Scala's compiler checks at *COMPILE* time that you're only ever putting `Int`s in `Int`-labeled drawers -- so a whole category of "wait, this was actually a string the whole time" bugs simply can't happen once your code compiles.



Why Long suffixes and explicit types matter in Spark code

When working with Spark, timestamps, row counts, and byte-offsets in large datasets routinely exceed `Int`'s roughly 2.1 billion range -- forgetting the `L` suffix on a large literal, or letting a computation silently overflow an `Int`, is a classic, hard-to-spot bug in real Big Data pipelines processing billions of rows. Being deliberate about `Long` vs. `Int` is not academic pedantry here; it's a genuine production concern.



Check yourself — Section 2

- Q1. Why must you write ``9000000000L`` instead of ``9000000000`` for a large `Long` value?
- Q2. What's the difference between Scala's `Unit` type and Java's `void`, conceptually?
- Q3. In your own words, why does Scala's `Nothing` type have no values, and what is it used for?

Hands-on Challenge:

Write a short Scala snippet declaring one value of each of these types: `Int`, `Long`, `Double`, `Boolean`, `Char`, and `String`, using s-interpolation to print all six values in a single formatted sentence.

3. Operators and Methods Used in Scala

Here is a genuinely distinctive Scala feature worth understanding early: OPERATORS ARE METHODS. When you write `2 + 3`, Scala is actually calling the `+` method defined on the `Int` object `2`, passing `3` as its argument -- it's exactly equivalent to writing `2.+(3)`. This single fact explains a great deal of Scala's syntax flexibility, including why you can define your own custom operators on your own classes.

3.1 Arithmetic, Relational, and Logical Operators

Category	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (addition, subtraction, multiplication, division, remainder)
Relational	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> (equality/comparison, all return Boolean)
Logical	<code>&&</code> <code> </code> <code>!</code> (AND, OR, NOT — <code>&&</code> and <code> </code> short-circuit)
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> <code>>>></code> (bitwise AND/OR/XOR/NOT and shifts)
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> (var only — val cannot be reassigned)

```
val a = 17
val b = 5
println(a + b)      // 22
println(a % b)      // 2   (remainder)
println(a / b)      // 3   (integer division -- truncates, not 3.4)
println(a.toDouble / b) // 3.4 (force floating-point division)

println(a > b && b > 0) // true -- && short-circuits: if left is false,
                        // the right side is never even evaluated
```

3.2 Operators Are Just Method Calls

```
// These two lines are 100% equivalent -- infix notation is just syntax sugar
val sum1 = 2 + 3
val sum2 = 2.+(3)
println(sum1 == sum2) // true

// Because operators are methods, YOU can define custom operators on your own
types
case class Vector2D(x: Double, y: Double) {
  def +(other: Vector2D): Vector2D = Vector2D(x + other.x, y + other.y)
```

```

    def *(scalar: Double): Vector2D = Vector2D(x * scalar, y * scalar)
  }

val v1 = Vector2D(1.0, 2.0)
val v2 = Vector2D(3.0, 4.0)
println(v1 + v2)          // Vector2D(4.0,6.0)  -- reads naturally, like built-in
                           // math
println(v1 * 2.5)         // Vector2D(2.5,5.0)

```

3.3 Common Methods on Core Types

Beyond operators, every core type carries a rich set of ordinary named methods. A handful you'll use constantly:

```

// String methods
val s = "  Apache Spark  "
println(s.trim)           // "Apache Spark"
println(s.trim.toLowerCase) // "apache spark"
println(s.trim.split(" ").toList) // List(Apache, Spark)
println(s.trim.startsWith("Apache")) // true
println(s.trim.replace("Spark", "Flink")) // "Apache Flink"

// Numeric methods
println((-7).abs)         // 7
println(7.max(12))        // 12
println(7.min(12))        // 7
println(3.14159.round)    // 3
println(9.toDouble.sqrt)  // NOTE: sqrt isn't on Double directly --
                           // use: math.sqrt(9) -> 3.0

```

3.4 Method Call Syntax — With and Without Dots/Parens

Scala allows a method taking exactly one argument to be called using INFIX notation (no dot, no parentheses), which is precisely what makes user-defined operators like the Vector2D example above read naturally. A method taking NO arguments can often be called without parentheses at all.

```

val text = "hello"

// These three calls are all equivalent
text.contains("ell")
text contains "ell"          // infix notation -- no dot, no parens

// A parameterless method -- both forms work, but convention prefers
// parens for methods with side effects, and no parens for pure "properties"
text.length                 // preferred -- reading .length feels like a property

```

```
text.length()    // also valid, but less idiomatic here
```



Everything Is Just a Message Being Sent

In Scala's mental model, $2 + 3$ isn't really 'plus-ing two numbers together' in some special built-in way -- it's the value 2 being sent a message called `+`, carrying the argument 3, and responding with a new value. This is the same idea Smalltalk (one of the earliest object-oriented languages) pioneered decades ago: everything, even something as basic as arithmetic, is just an object responding to a message. Once this clicks, defining your own `+` on `Vector2D` stops looking like a special trick and starts looking like the most natural thing in the world -- you're just teaching a new kind of object how to respond to a familiar message.



Where this shows up directly in Spark code

Spark's `DataFrame` API leans heavily on this exact feature. Expressions like `df("age") > 18` or `col("price") * col("quantity")` are only possible to write this naturally because Spark's `Column` class defines custom `>` and `*` operators using precisely the method-call mechanism covered in this section -- the `DataFrame` API's readable, SQL-like syntax is a direct, deliberate application of Scala's operators-are-methods design.



Check yourself — Section 3

- Q1. Rewrite `10 - 4` as an explicit method call, without using the `-` operator symbol.
- Q2. Why does `&&` "short-circuit," and give one practical reason that behavior is useful.
- Q3. What specific Scala feature makes it possible to write `df("age") > 18` naturally in Spark's `DataFrame` API?

Hands-on Challenge:

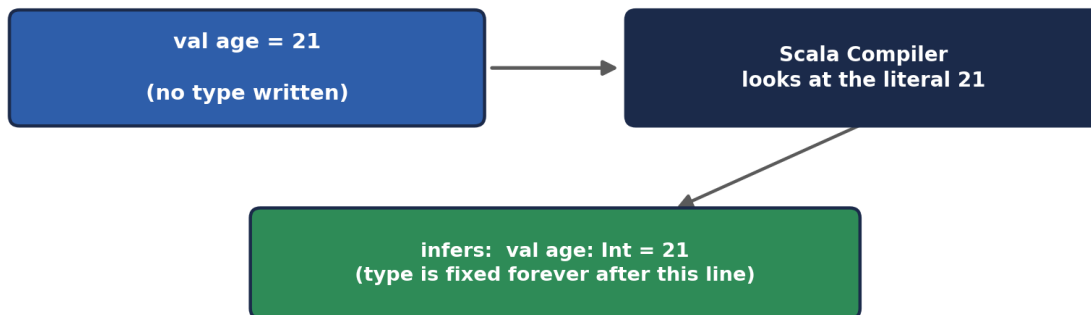
Define your own case class `Money(cents: Int)` representing a monetary value, and implement custom `+` and `>` operators on it so that `Money(150) + Money(250)` produces `Money(400)`, and `Money(500) > Money(300)` evaluates to `true`.

4. Introduction to Type Inference

Section 1.3 mentioned that Scala is statically typed but rarely feels verbose -- type inference is the specific mechanism that makes this possible. The compiler analyzes the right-hand side of a declaration (or an expression's context) and determines its type automatically, without you writing it explicitly.

4.1 How It Works

Type Inference — The Compiler Figures Out the Type For You



Type Inference — The Compiler Figures Out the Type For You

It's critical to understand what type inference is NOT: it does not mean Scala is dynamically typed, and it does not mean the type can change later. Once the compiler infers age to be an Int, that variable is an Int for its entire lifetime, exactly as if you had written the type explicitly -- inference only saves you from TYPING the type annotation; it doesn't remove the type itself.

```

val age = 21           // inferred: Int
val price = 19.99      // inferred: Double
val name = "Spark"     // inferred: String
val active = true      // inferred: Boolean

// You CAN still write the type explicitly -- inference is optional, not
// mandatory
val explicitAge: Int = 21

// But once inferred (or declared), the type is LOCKED
var count = 10         // inferred: Int
count = 15             // fine -- still an Int
  
```

```
count = "fifteen"    // COMPILER ERROR: type mismatch; found String, required
Int
```

4.2 Type Inference for Function Return Types

Scala can also infer a method's RETURN type from its body, though many teams adopt a style guideline requiring explicit return types on public methods for readability and to avoid accidentally changing a public API's type when refactoring the method body.

```
// Return type Int is inferred from the body
def square(x: Int) = x * x

// Equivalent, with the return type made explicit (generally the safer style
// for any method other people/other code will call)
def squareExplicit(x: Int): Int = x * x

// Recursive functions are the ONE case where Scala CANNOT infer the return
// type -- you must annotate it explicitly
def factorial(n: Int): Int =
  if (n <= 1) 1 else n * factorial(n - 1)
```

4.3 Type Inference With Generics and Collections

```
// The element type of a collection is inferred from its contents
val numbers = List(1, 2, 3)    // inferred: List[Int]
val names = List("Aditi", "Rahul") // inferred: List[String]

// When elements have a common supertype, Scala infers THAT supertype
val mixed = List(1, 2.5, 3)    // inferred: List[Double]
                                // (Int values are widened to Double)

// map() and other higher-order functions infer their OWN return type
// from the function passed to them
val doubled = numbers.map(n => n * 2)    // inferred: List[Int]
val strings = numbers.map(n => n.toString) // inferred: List[String]
```

A genuine limitation to be aware of

Type inference in Scala works LOCALLY and left-to-right; it cannot infer a parameter's type from how it's used LATER in a function body, and it cannot infer the types of a method's PARAMETERS at all (only its return type) -- parameters must always be explicitly typed. This is different from some languages with more powerful, whole-program type inference (like Haskell), and is a deliberate trade-off Scala makes for faster compilation and more locally-understandable error messages.



The Tailor Who Reads the Fabric

Explicit typing is like handing a tailor a precise written spec: "this shirt must be exactly size Medium." Type inference is a skilled tailor who looks at the fabric you handed them (the value on the right of the =) and correctly determines the size themselves -- fast, convenient, and just as precise as the written spec, because they're not guessing, they're reading exact information right in front of them.

But that tailor still can't read your mind about a garment you haven't shown them yet -- which is exactly why Scala can't infer a function PARAMETER'S type (you haven't handed over any fabric yet when you're just describing the function's shape) even though it happily infers a val's type from the value you did hand over.



Why this matters for Spark code specifically

Spark's Scala API leans heavily on type inference to keep DataFrame and RDD transformation chains readable: `df.filter(col("age") > 18).select("name")` never requires you to annotate intermediate types, because each step's return type is inferred from the previous step. Understanding inference is what lets you correctly read a Spark stack trace that references an inferred type you never explicitly wrote in your own code.



Check yourself — Section 4

Q1. Does type inference make Scala dynamically typed? Explain why or why not.

Q2. Why does Scala require an explicit return type annotation on recursive functions specifically?

Q3. Can Scala infer the type of a function's PARAMETERS? Why or why not?

Hands-on Challenge:

Write five `val` declarations with no type annotations, covering an `Int`, a `Double`, a `String`, a `Boolean`, and a `List` of your choosing, then write down what you believe the compiler infers for each — and if you have Scala available, verify using `:type` in the Scala REPL (e.g., `scala> :type 21`).

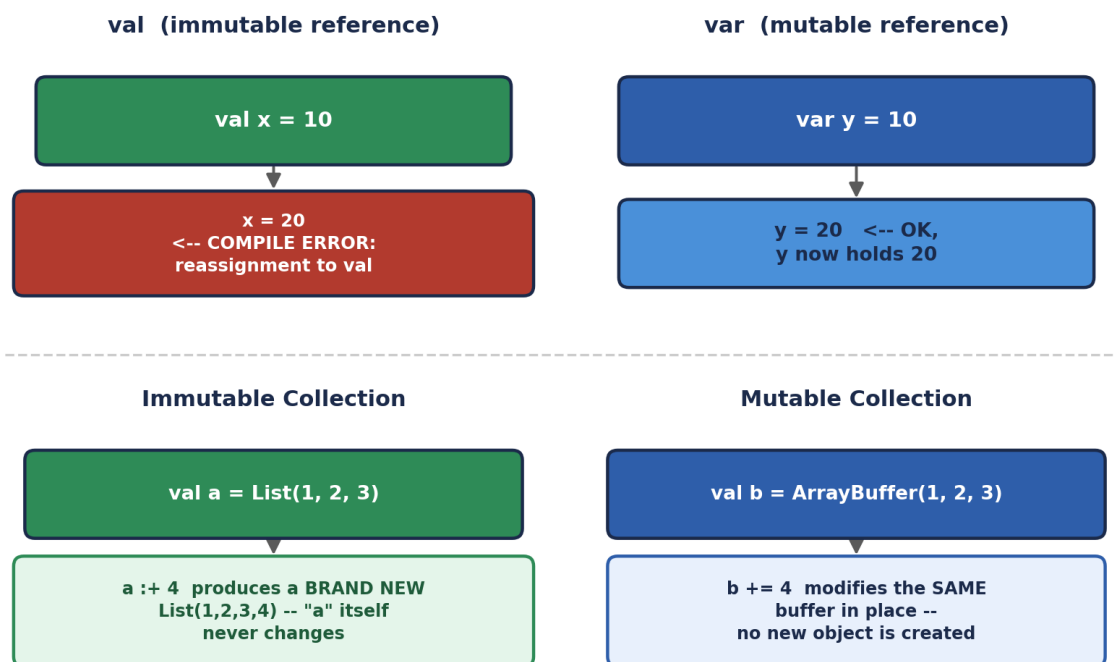
5. Mutable vs. Immutable Collections

This distinction is arguably the single most important conceptual thread running through idiomatic Scala, and it connects directly forward to why Spark's RDDs are immutable (a design choice Unit III will explore in depth). This section covers both the `val`/`var` distinction for simple variables and the mutable/immutable distinction for entire collections.

5.1 `val` vs. `var` — Immutable and Mutable References

A `val` is a reference that, once assigned, can NEVER be reassigned to point at a different value -- attempting to do so is a compile-time error, not a runtime one. A `var` CAN be reassigned. Idiomatic Scala style strongly prefers `val` by default, reaching for `var` only when mutation is genuinely the clearest way to express an algorithm.

`val` vs. `var`, and Immutable vs. Mutable Collections



val vs. var, and Immutable vs. Mutable Collections

```
val x = 10
x = 20      // COMPILE ERROR: reassignment to val

var y = 10
y = 20      // fine -- y now holds 20
y += 5      // fine -- y now holds 25
```

5.2 Immutable Collections — the Default in Scala

Scala's DEFAULT collection types (List, Vector, Map, Set, when imported with no special package) are all IMMUTABLE: once created, a collection's contents never change. Operations that look like they "modify" a collection -- appending an element, removing an element -- actually return a BRAND NEW collection, leaving the original completely untouched.

```
val original = List(1, 2, 3)
val withFour = original :+ 4      // append -- returns a NEW list

println(original)    // List(1, 2, 3)    -- UNCHANGED
println(withFour)    // List(1, 2, 3, 4) -- a separate, new list

val map1 = Map("a" -> 1, "b" -> 2)
val map2 = map1 + ("c" -> 3)      // returns a NEW map
println(map1)           // Map(a -> 1, b -> 2)    -- UNCHANGED
println(map2)           // Map(a -> 1, b -> 2, c -> 3) -- a separate, new map
```

5.3 Mutable Collections — Opt-In, Explicit

For situations where in-place mutation genuinely IS the right tool (performance-critical inner loops building up a large collection, for instance), Scala provides mutable counterparts under the `scala.collection.mutable` package, which must be imported explicitly -- this explicit-import requirement is a deliberate design choice: mutability should always be a visible, deliberate decision in the code, never an accidental default.

```
import scala.collection.mutable.ArrayBuffer

val buffer = ArrayBuffer(1, 2, 3)
buffer += 4      // MODIFIES buffer in place -- no new object
println(buffer)  // ArrayBuffer(1, 2, 3, 4)

import scala.collection.mutable.{Map => MutableMap}
val scores = MutableMap("Aditi" -> 90)
scores("Aditi") = 95    // MODIFIES the existing map in place
scores("Rahul") = 88    // adds a new entry, in place
println(scores)        // MutableMap(Aditi -> 95, Rahul -> 88)
```

5.4 Why Immutability Is the Default, Not an Afterthought

- Thread safety — an immutable object can never be corrupted by two threads modifying it simultaneously, because no thread can ever modify it at all. This eliminates an entire category of concurrency bugs by construction, without needing locks.

- Reasoning about code — a `val`'s value is guaranteed never to change after its declaration, so understanding what a piece of code does never requires mentally tracking "but what if this got mutated somewhere else first."
- Safe sharing — an immutable collection can be freely shared across function calls, across threads, or across machines in a distributed system, with zero risk that one consumer's use of it affects another's — which is EXACTLY why Spark's RDDs are immutable: a partition can be safely cached, shared across tasks, and sent to multiple executors without any of them being able to corrupt it for the others.

The direct line from this section to Spark's core design

Every Spark RDD, once created, is immutable — transformations like ``map()`` and ``filter()`` never modify an existing RDD; they always return a NEW RDD, exactly like ``original :+ 4`` above returns a new List. This isn't a coincidence: Spark's creators, working in Scala, deliberately extended Scala's immutable-collection philosophy to Spark's own distributed data structures, for exactly the thread-safety and safe-sharing reasons listed above, now applied across an entire cluster instead of just within one JVM.

A Photograph vs. a Whiteboard

An immutable collection is a photograph -- once printed, it captures a specific moment forever; if you want a different picture, you take an entirely NEW photograph (a new collection), and the old one sits untouched in the album. A mutable collection is a whiteboard -- erase part of it and write something new, and the SAME whiteboard now shows different content; anyone else looking at that whiteboard right now sees the change too, immediately.

The photograph approach means you can hand out copies to a hundred people and know none of them can ever surprise you by handing back a photo that mysteriously changed. The whiteboard approach means if two people try to write on it at the same moment, you can end up with a smeared, half-overwritten mess -- exactly the class of bug immutability sidesteps entirely.

A common beginner trap

``val list = List(1,2,3)`` makes the REFERENCE ``list`` unassignable, not the CONTENTS mutable-or-not — List is already immutable regardless of `val/var`. But ``var buffer = ArrayBuffer(1,2,3)`` and ``val buffer2 = ArrayBuffer(1,2,3)`` behave differently in a subtler way: with ``val buffer2``, you cannot reassign ``buffer2`` to a whole new `ArrayBuffer`, but you absolutely CAN still call ``buffer2 += 4`` to mutate its contents in place — `val` only locks the reference, never the mutability of what it points to.

Check yourself — Section 5

Q1. What's the difference between ``val x = 10; x = 20`` failing, and ``val list = ArrayBuffer(1,2,3); list += 4`` succeeding? Aren't they both "changing" something bound with `val`?

Q2. Name two concrete engineering benefits of immutability, beyond "it's the Scala convention."

Q3. How does Spark's RDD design directly inherit Scala's immutable-collection philosophy, and why does that matter for distributed computing specifically?

Hands-on Challenge:

Write two versions of a function that removes all negative numbers from a collection of integers: one using an immutable List (returning a new filtered list), and one using a mutable ArrayBuffer (removing elements in place with `--` or `filterInPlace`). Compare the two implementations and note which one you find easier to reason about, and why.

6. Functions in Scala

Section 1.2 mentioned that Scala treats functions as first-class values. This section unpacks exactly what that means in practice, moving from ordinary named methods through anonymous functions, higher-order functions, and currying.

6.1 Defining Functions with def

```
def add(a: Int, b: Int): Int = a + b
println(add(3, 4))    // 7

// Default parameter values
def greet(name: String, greeting: String = "Hello"): String = s"$greeting, $name!"
println(greet("Aditi"))           // Hello, Aditi!
println(greet("Rahul", "Hey"))    // Hey, Rahul!

// Named arguments -- order stops mattering
println(greet(greeting = "Hi", name = "Meera"))    // Hi, Meera!

// Variable-length argument lists (varargs)
def sumAll(numbers: Int*): Int = numbers.sum
println(sumAll(1, 2, 3, 4, 5))    // 15
println(sumAll())                 // 0
```

6.2 Anonymous Functions (Lambdas)

The Function Spectrum in Scala

From a Named Method to a Higher-Order, Curried Function

def square(x: Int): Int = x * x	A plain named method
val square = (x: Int) => x * x	The same logic as an anonymous function VALUE
list.map(x => x * x)	Passed directly to a HIGHER-ORDER function (map)
def adder(a: Int)(b: Int) = a + b val add5 = adder(5)_	CURRIED -- returns a new

The Function Spectrum in Scala

An anonymous function -- often called a lambda -- is a function VALUE with no name, defined inline wherever it's needed. Scala's syntax for this is (parameters) => expression, and it can be assigned to a val exactly like any other value, per Section 1.2's "functions are first-class values" feature.

```
// A named function value (not a def -- a val holding a function)
val square: Int => Int = x => x * x
println(square(5))    // 25

// The SAME logic, defined completely anonymously and used immediately
println((x: Int) => x * x)(5)    // 25

// Anonymous functions passed directly as arguments -- extremely common
val numbers = List(1, 2, 3, 4, 5)
val squared = numbers.map(x => x * x)
println(squared)    // List(1, 4, 9, 16, 25)

// Scala's underscore shorthand for very short anonymous functions
val squaredShort = numbers.map(_ * _)    // WRONG -- needs two args; see below
val squaredCorrect = numbers.map(x => x * x)    // keep explicit for clarity here
val doubled = numbers.map(_ * 2)    // RIGHT -- one underscore, one param
```

6.3 Higher-Order Functions

A higher-order function is any function that either accepts another function as a parameter, or returns a function as its result (or both). This is the mechanism underneath nearly every Spark transformation you'll write: `map()`, `filter()`, and `reduce()` are all higher-order functions defined on Spark's RDD and DataFrame classes.

```
// A higher-order function: applyTwice ACCEPTS a function as a parameter
def applyTwice(f: Int => Int, x: Int): Int = f(f(x))

println(applyTwice(square, 3))    // square(square(3)) = square(9) = 81
println(applyTwice(x => x + 10, 3))    // (3+10)+10 = 23

// The built-in higher-order functions you'll use constantly on collections
val nums = List(1, 2, 3, 4, 5, 6)
println(nums.filter(x => x % 2 == 0))    // List(2, 4, 6)
println(nums.map(x => x * 10))    // List(10,20,30,40,50,60)
println(nums.reduce((a, b) => a + b))    // 21
println(nums.fold(100)((a, b) => a + b))    // 121 (100 + sum of nums)
println(nums.exists(x => x > 5))    // true
println(nums.forall(x => x > 0))    // true
println(nums.groupBy(x => x % 2 == 0))    // Map(false -> List(1,3,5), true -> List(2,4,6))
```

A function that RETURNS a function is the other half of "higher-order":

```
// multiplier RETURNS a function -- it doesn't compute a number itself,
// it builds and hands back a specialized function
def multiplier(factor: Int): Int => Int = {
  x => x * factor
}

val triple = multiplier(3)    // triple is now a function: Int => Int
val double = multiplier(2)
println(triple(7))           // 21
println(double(7))           // 14
```

6.4 Currying

Currying transforms a function that takes multiple arguments into a chain of functions, each taking a single argument, where calling it with fewer arguments than it needs returns a new function awaiting the rest. Scala supports this directly with multiple parameter lists.

```
// A normal two-argument function
def addNormal(a: Int, b: Int): Int = a + b

// The SAME logic, curried -- two parameter lists instead of one
def addCurried(a: Int)(b: Int): Int = a + b

println(addCurried(5)(3))    // 8 -- called with both argument lists at once

// The real power: PARTIAL APPLICATION -- supply only the first list,
// and get back a new, specialized function awaiting the second
val add5: Int => Int = addCurried(5)
println(add5(3))              // 8
println(add5(100))            // 105
println(add5(-5))             // 0

// A practical example: building specialized validators from one general
// function
def isInRange(min: Int)(max: Int)(value: Int): Boolean = value >= min && value
  <= max
val isValidPercentage = isInRange(0)(100) _
println(isValidPercentage(85)) // true
println(isValidPercentage(150)) // false
```

6.5 Recursion

Because Scala's functional side favors expressing repetition through function calls rather than mutable loop counters, recursion appears constantly in idiomatic Scala code -- including, importantly, Scala's tail-call optimization, which lets certain recursive functions run in constant stack space, exactly as efficiently as a loop.


```
// Ordinary recursion
def factorial(n: Int): Int =
  if (n <= 1) 1 else n * factorial(n - 1)
println(factorial(5))    // 120

// TAIL-RECURSIVE version -- the recursive call is the VERY LAST operation,
// which lets the compiler optimize it into a loop internally (no stack growth)
import scala.annotation.tailrec

@tailrec
def factorialTailRec(n: Int, accumulator: Int = 1): Int =
  if (n <= 1) accumulator
  else factorialTailRec(n - 1, n * accumulator)

println(factorialTailRec(5))    // 120
// @tailrec makes the compiler ERROR if the function ISN'T actually
// tail-recursive -- a useful safety net against accidental stack overflows
```



The Relay Race Baton vs. the Round-Trip Errand

Ordinary recursion is a round-trip errand: `factorial(5)` sends a runner to fetch `factorial(4)`, who sends a runner to fetch `factorial(3)`, and so on -- and every runner has to physically WAIT AROUND for their sub-runner to come back before they can multiply the result and report back up the chain themselves. Each waiting runner takes up a spot (a stack frame); for a deep enough chain, you run out of room (a stack overflow).

A tail-recursive call is a relay race baton hand-off: `factorialTailRec` doesn't wait around for anything -- its very last action IS the recursive call, carrying the running total (the accumulator) forward like a baton. Because nobody's waiting for a sub-result to come back, the compiler can recognize this pattern and reuse the SAME spot on the track for every leg of the race, turning what looks like recursion into, mechanically, a simple loop.



Why higher-order functions and currying matter directly for Spark

Every single Spark transformation you'll write in this course -- ``rdd.map(...)``, ``rdd.filter(...)``, ``rdd.reduce(...)`` -- is you calling a higher-order function and handing it an anonymous function, using exactly the syntax from Section 6.2 and 6.3. Currying specifically shows up in Spark's own API design too: methods like ``foldLeft`` take a curried signature ``(zeroValue: B)(combiningFunction: (B, A) => B)`` for the exact same reason shown in Section 6.4 -- letting Scala's type inference use the type of the zero value to correctly infer the combining function's types.



Check yourself — Section 6

Q1. What does it mean for a function to be "higher-order," and give one example from Scala's built-in collection methods.

Q2. Walk through what `addCurried(5)` returns (not `addCurried(5)(3)` — just `addCurried(5)` on its own), and why that's useful.

Q3. Why can a tail-recursive function run in constant stack space, while an ordinary recursive function cannot?

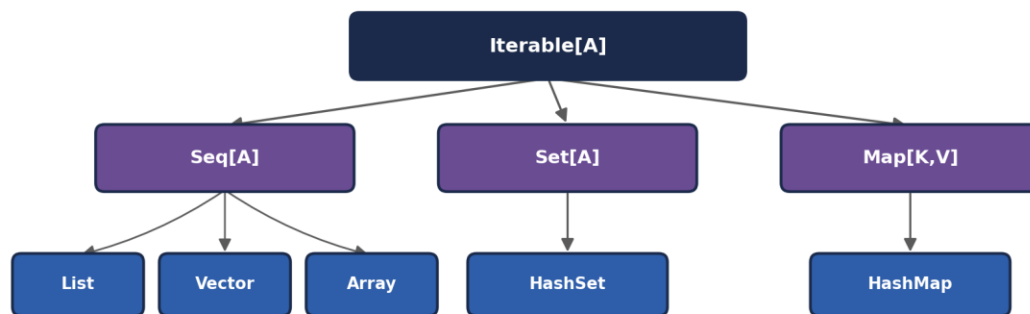
Hands-on Challenge:

Write a curried function `def clamp(min: Int)(max: Int)(value: Int): Int` that returns `value`, but forced into the `[min, max]` range (i.e., returns `min` if value is too low, `max` if too high, otherwise `value` unchanged). Use partial application to create a specialized `val clampToPercentage = clamp(0)(100) _`, then test it against several values.

7. Lists, Maps, and Streams in Scala

This closing section covers the three collection types the syllabus names specifically, each in depth: List (an ordered, linked sequence), Map (key-value pairs), and Stream/LazyList (a lazily-evaluated, potentially infinite sequence). All three connect directly back to Section 5's mutable/immutable distinction — every example below uses the immutable default unless stated otherwise.

The Scala Collections Hierarchy



ArrayBuffer, LinkedHashSet, TreeMap, and others follow the same pattern

Every one of these exists in TWO flavors:

`scala.collection.immutable`
(the DEFAULT import)

`scala.collection.mutable`
(must be imported explicitly)

The Scala Collections Hierarchy

7.1 List — an Ordered, Immutable Linked Sequence

A Scala List is implemented internally as a singly-linked list: an empty list Nil, or a head element joined to a tail list via the :: ("cons") operator. This internal structure is why prepending an element to the front of a List is extremely cheap (constant time), while appending to the END or accessing an arbitrary index by position is comparatively expensive (linear time) -- a detail that matters when choosing between List and Vector for performance-sensitive code.

```

val empty = Nil // the empty list
val fruits = List("apple", "banana", "cherry")

// Building a list the "cons" way -- exactly how List is built internally
val built = "apple" :: "banana" :: "cherry" :: Nil
println(built) // List(apple, banana, cherry)
println(built == fruits) // true -- same contents, same structure
  
```

```
// Core operations
println(fruits.head)           // apple -- the first element
println(fruits.tail)          // List(banana, cherry) -- everything after head
println(fruits.isEmpty)       // false
println(fruits.length)        // 3
println(fruits(1))            // banana -- indexed access (0-based)
println(fruits.reverse)       // List(cherry, banana, apple)
println(fruits.contains("banana")) // true

// Adding elements -- ALWAYS returns a new list (Section 5.2)
println("mango" :: fruits)     // List(mango, apple, banana, cherry) --
// prepend, O(1)
println(fruits :+ "mango")     // List(apple, banana, cherry, mango) --
// append, O(n)
println(fruits ++ List("kiwi", "fig")) // concatenate two lists

// The higher-order functions from Section 6.3, applied to a real list
println(fruits.map(_.toUpperCase)) // List(APPLE, BANANA, CHERRY)
println(fruits.filter(_.length > 5)) // List(banana, cherry)
println(fruits.sortBy(_.length))    // List(apple, banana, cherry) -> by
// length
println(fruits.sorted)              // alphabetical order
```

List also supports pattern matching directly, connecting back to Section 1.5:

```
def describeList(list: List[Int]): String = list match {
  case Nil          => "empty list"
  case head :: Nil   => s"single element: $head"
  case head :: tail  => s"starts with $head, followed by ${tail.length}
more"
}
println(describeList(List()))           // empty list
println(describeList(List(5)))          // single element: 5
println(describeList(List(1,2,3,4)))    // starts with 1, followed by 3 more
```

7.2 Map — Key-Value Pairs

A Scala Map associates unique keys with values, exactly like a dictionary or hash table in other languages. As with List, the default Map (`scala.collection.immutable.Map`) returns a new map from every "modifying" operation.

```
val studentGrades = Map("Aditi" -> 92, "Rahul" -> 85, "Meera" -> 78)
// note: "Aditi" -> 92 is itself shorthand for the tuple ("Aditi", 92)

// Looking up values
println(studentGrades("Aditi"))           // 92
println(studentGrades.get("Aditi"))       // Some(92) -- safe lookup, see
// below
```

```
println(studentGrades.get("Zoya"))           // None -- key doesn't exist, no
crash
println(studentGrades.getOrElse("Zoya", 0))  // 0 -- provide a fallback
println(studentGrades.contains("Rahul"))     // true

// Adding / removing -- ALWAYS returns a new map
val updated = studentGrades + ("Zoya" -> 88)
val removed = studentGrades - "Rahul"
println(updated)    // Map(Aditi -> 92, Rahul -> 85, Meera -> 78, Zoya -> 88)
println(removed)    // Map(Aditi -> 92, Meera -> 78)

// Iterating and transforming
studentGrades.foreach { case (name, grade) => println(s"$name scored $grade") }

val passed = studentGrades.filter { case (_, grade) => grade >= 80 }
println(passed)    // Map(Aditi -> 92, Rahul -> 85)

val curved = studentGrades.map { case (name, grade) => (name, grade + 5) }
println(curved)    // Map(Aditi -> 97, Rahul -> 90, Meera -> 83)

println(studentGrades.keys)    // Set of all names
println(studentGrades.values)  // collection of all grades
```

☑ Map.get() and Option -- a preview of safer null-handling

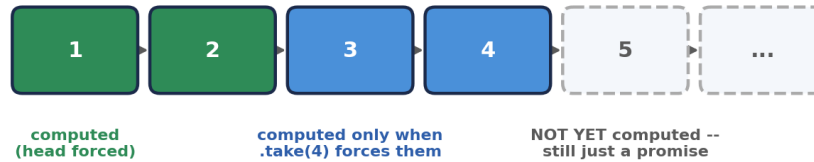
Notice `studentGrades.get("Zoya")` returns `None` instead of crashing or returning `null` — Scala's `Option` type (`Some(value)` or `None`) is the idiomatic way to represent "a value might not be there," used throughout Scala's standard library instead of returning `null`. You'll use `Option` constantly once you start working with Spark's own APIs, many of which return `Option`-wrapped results for exactly this reason.

7.3 Stream / LazyList — Lazily Evaluated Sequences

A `Stream` (renamed `LazyList` starting in Scala 2.13, with the same core concept) is a sequence whose elements are computed **ON DEMAND**, one at a time, rather than all at once when the collection is created. This makes it possible to represent conceptually **INFINITE** sequences, since only the elements you actually ask for are ever computed.

Streams / LazyList — Computed On Demand, Element by Element

`val nats = LazyList.from(1) // a conceptually INFINITE stream`



```
nats.take(4).toList
// forces exactly 4 elements -- 5, 6, 7, ... are never touched
```

Only as much of the stream as you actually ask for is ever computed

Streams / LazyList — Computed On Demand, Element by Element

```
// An infinite stream of all natural numbers starting from 1
val nats: LazyList[Int] = LazyList.from(1)

// Nothing has been computed yet beyond what's needed to represent "1, then
// more"
println(nats.take(5).toList)    // List(1, 2, 3, 4, 5)
// Only the first 5 elements were EVER actually computed

// Building a LazyList with an explicit recursive definition
def fibonacci(a: Int, b: Int): LazyList[Int] = a #:: fibonacci(b, a + b)
val fibs = fibonacci(0, 1)
println(fibs.take(10).toList)   // List(0, 1, 1, 2, 3, 5, 8, 13, 21, 34)

// map() and filter() on a LazyList are THEMSELVES lazy -- they build up
// a chain of promises, not actual computed values, until something forces them
val evenSquares = nats.map(x => x * x).filter(_ % 2 == 0)
println(evenSquares.take(4).toList) // List(4, 16, 36, 64)
```

This laziness is directly analogous to a concept Unit I already introduced: Spark's own lazy evaluation of transformations. A LazyList's `map()` and `filter()` calls build up a chain of unevaluated operations exactly the way Spark's RDD transformations build up a DAG -- and in both cases, nothing actually computes until something "forces" the result (calling `.toList` or `.take(n)` on a LazyList; calling an action like `.collect()` or `.count()` on an RDD).



The direct conceptual bridge to Spark's execution model

This is not a coincidence, and it's worth sitting with: Spark's RDD lineage graph and Scala's LazyList share the exact same underlying idea -- describe a computation as a chain of steps first, and only actually RUN it when a concrete result is genuinely needed. Scala's language-level laziness (LazyList, and more generally the lazy keyword on any val) is one of the conceptual building blocks that made Spark's own lazy DAG execution model a natural design choice for its creators, who were, after all, thinking in Scala the whole time.



The Never-Ending Buffet That Only Cooks What You Order

An ordinary List is a buffet where every dish is pre-cooked and laid out before you arrive -- fine for a fixed, finite menu, but impossible if the menu is meant to be infinite. A LazyList is a buffet with an infinite menu and a chef standing behind the counter who only actually COOKS a dish the moment you point at it. You can stand there theoretically browsing an endless menu forever, and the chef never breaks a sweat, because their real work only happens exactly as fast as you actually order (call `.take(n)`, or otherwise force elements).



Check yourself — Section 7

- Q1. Why is prepending to the front of a List fast, while appending to the end is comparatively slow?
- Q2. What does `studentGrades.get("Zoya")`` return if "Zoya" isn't a key in the map, and why is that safer than throwing an exception?
- Q3. In your own words, explain the conceptual similarity between a Scala LazyList and a Spark RDD's lineage-based lazy execution.

Hands-on Challenge:

Using a `Map[String, Int]` representing at least 5 products and their prices, write code that: (1) adds one new product, (2) filters to only products under a certain price, (3) computes the total price of all products using a fold or sum, and (4) prints a formatted summary line for each remaining product using s-interpolation.

Unit II Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. Why does ``2 + 3`` in Scala actually amount to a method call, and what does that fact make possible?
2. Q2. What's the practical difference between ``val list = List(1,2,3)`` and ``val buffer = ArrayBuffer(1,2,3)`` when it comes to modifying contents?
3. Q3. What does currying a function let you do that a normal multi-argument function doesn't easily allow?

☒ Answer key (cover before peeking)

A1: It's exactly equivalent to ``2.+(3)`` — the `+` symbol is just infix syntax for calling the `+` method defined on `Int`. This makes it possible to define custom operators on your own types (like `Vector2D`'s `+` and `*`), which is exactly how Spark's `Column` class supports expressions like ``col("age") > 18``.

A2: ``list`` is immutable — there is no way to change its contents; any "modification" (like ``:+``) returns a brand-new `List`, leaving the original untouched. ``buffer`` is a mutable `ArrayBuffer` — calling ``buffer += 4`` changes the SAME object in place, with no new object created.

A3: Currying enables partial application — supplying only some of a curried function's arguments returns a new, specialized function awaiting the rest (e.g., ``multiplier(5)`` returning a reusable "multiply by 5" function). A normal multi-argument function requires all arguments at once, every time.

Hands-On Exercise

Complete, end to end, using the Scala REPL (``scala`` at your terminal), Scastie (an online REPL), or a ``.scala`` script file run with ``scala myfile.scala``:

4. Define a case class ``Product(name: String, price: Double, category: String)`` (Section 1.4/1.5) and create an immutable ``List[Product]`` (Section 7.1) with at least 6 products across at least 3 categories.
5. Using ``groupBy`` (Section 6.3), group the products by category into a ``Map[String, List[Product]]`` (Section 7.2).
6. For each category, use ``map`` and ``reduce`/`fold`` (Section 6.3) to compute the total price of products in that category, producing a ``Map[String, Double]``.
7. Write a curried function ``def applyDiscount(percent: Double)(price: Double): Double`` (Section 6.4) that reduces a price by the given percentage, and use partial application to create a ``val tenPercentOff`` you apply to every product's price using ``map``.
8. Finally, build a ``LazyList`` (Section 7.3) that generates an infinite sequence of restock reminder days (every 7 days starting from day 1), and print only the first 5 reminder days using ``.take(5).toList``.

** What this exercise is really testing**

This single exercise deliberately exercises nearly every syllabus topic in this unit at once: case classes and pattern-friendly data modeling (Section 1), immutable List/Map manipulation (Section 5 and 7), higher-order functions like groupBy/map/reduce (Section 6.3), currying and partial application (Section 6.4), and lazy, on-demand computation via LazyList (Section 7.3) — the same lazy-evaluation idea that underlies Spark's own execution model, which you'll meet again directly in later units.